

day01 - 原理介绍（一）

RAG项目资料具体地址：

链接: https://pan.baidu.com/s/1SAiB2DE_5yiJ4EXMizu59A 提取码: 2uxv

一、RAG 简介

1.1 大模型的局限性

LLM 虽然强大，但是也存在一些很难根除的“疾病”：

1) 知识时效性

LLM 因其具有海量参数，需要花费相当的物力与时间成本进行预训练和微调，同时商用 LLM 还需要进行各种安全测试与风险评估等。因此 LLM 会存在一定的知识滞后问题，无法回答训练时间点之后相关的问题。

2) 领域知识缺失

LLM 掌握了大量通用知识。但是在一些细分小众专业领域，可能存在很少或者完全没有训练数据。因此 LLM 无法掌握或完全不了解该领域相关的知识，也就无法准确回答这些领域相关的问题。

3) 无法确定信息来源

LLM 在生成回答时，无法确定其回答内容的依据，也就无法确定其回答的可靠性。

4) 幻觉

LLM 在生成回答时，可能会一本正经的胡说八道，这被称为 LLM 的“幻觉”问题。“幻觉”可以体现为错误陈述或编造事实、错误的复杂推理或者在复杂语境下理解能力不足等。

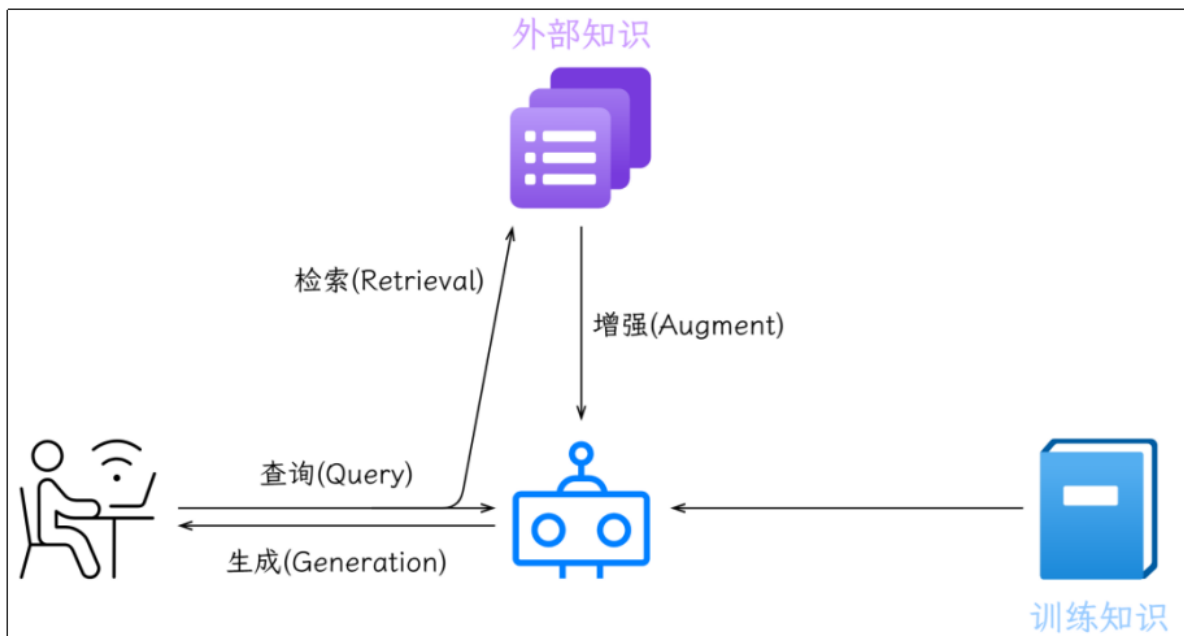
“幻觉”产生的原因可能是训练知识存在偏差，这些信息被 LLM 学习后在输出中复现；或者是 LLM 训练时过度泛化，将普通的模式应用在特定场合导致不准确输出；也可能是 LLM 本身没有真正学习到训练数据中深层次的含义，导致在一些需要深入理解或复杂推理的任务中出错；还有可能是 LLM 缺乏某些领域的相关知识，在面临这些领域的相关问题时编造不存在的信息。

因此，LLM 的强大并不能掩盖其当前仍存在的缺陷，但也正是这些不足促发着人工智能不断发展，比如研发更先进的模型架构、训练技术；使用更高效的数据清洗与预处理方法获取更高质量的训练数据；使用更先进的方法提升模型输出的可解释性等。

1.2 RAG 简介

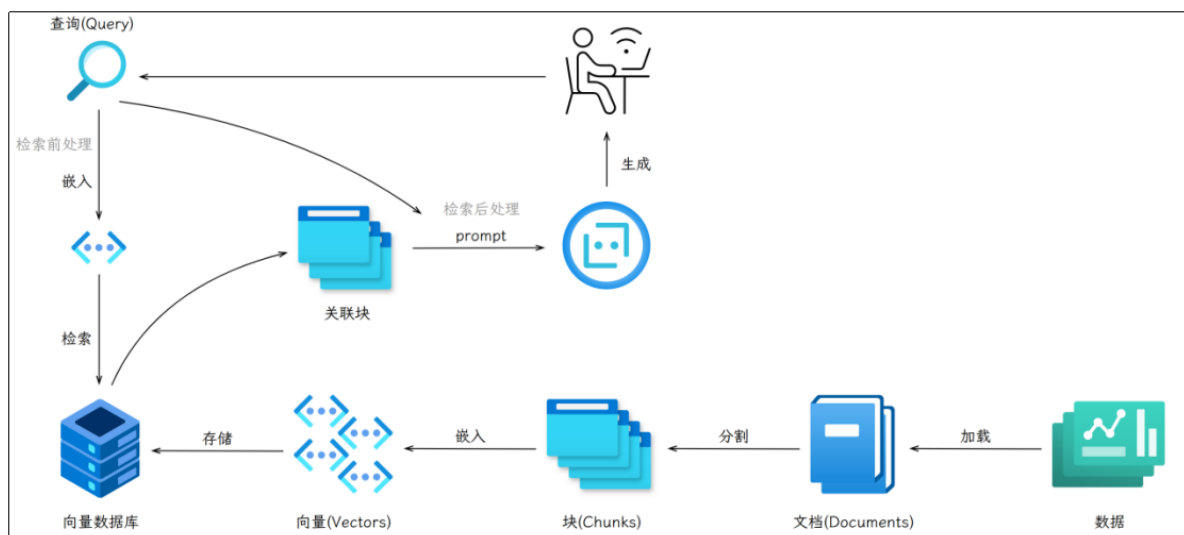
为了改善 LLM 在时效性、可靠性与准确性方面的不足，以便让其能够在更多场景下发挥作用，各种针对 LLM 优化的方法应运而生。RAG (Retrieval-Augmented Generation, 检索增强生成) 就是其中一种被广泛研究和应用的优化架构。

RAG 的基本思想为：将传统的生成式 LLM 和实时信息检索技术相结合，为 LLM 补充来自外部的相关数据和上下文，来帮助 LLM 生成更加准确可靠的内容。这使得 LLM 在生成内容时可以依赖实时与个性化的数据和知识，而非仅仅依赖训练知识。



借助 RAG，LLM 能够在生成时利用更广泛、更深入、更新、更准确的外部知识；并且能够更高效地利用计算资源在短期内提升 LLM 输出能力，避免直接扩展 LLM 参数或以更高的成本和更长的时间去训练 LLM。RAG 就相当于在 LLM 回答时给它一本参考书。

接下来我们从技术层面来看一个最基础、最常见的 RAG 应用的逻辑架构与流程。



通常情况下，可以将一个RAG应用分为数据索引（Indexing）和数据查询（Query）两个阶段：

1.2.1 数据索引阶段

RAG的核心之一是通过检索来增强生成，那么首先就需要准备可以检索的内容。传统计算机检索技术通常使用的是关键词检索，通过关键词的匹配程度来对知识库中的信息进行精确或模糊的检索，计算相关性并排序输出。但是在RAG应用中，最常见的检索方式是使用向量的语义检索来获取相关数据块，因此向量存储索引是RAG应用中最常见的索引形式。

数据索引阶段通常包含以下几个关键阶段：

- (1) **加载 (Loading)**：RAG应用需要的知识数据可能以不同形式不同格式存在，可能是结构化、半结构化、非结构化的数据，可能是PDF、HTML、Word或Markdown等格式，需要对这些数据统一进行处理和解析，将其转换为能够读取和使用的数据。
- (2) **分割 (Splitting)**：为了能够更精确地检索到相关数据，需要将一个较大的内容分割成一个个小的知识块。
- (3) **嵌入 (Embedding)**：分割成一个个知识块之后，需要对知识块进行嵌入，将知识块转换为向量。

(4) **索引 (Indexing)**：将向量化之后的知识块存储到向量数据库中。向量数据库会提供强大的向量检索算法与管理接口，后续可以方便地对输入问题进行语义检索。不过索引形式也并非只有向量数据库索引这一种，很多应用也会根据自身需要来构造其他形式的索引，如知识图谱索引、关键词表索引等。

1.2.2 数据查询阶段

在数据索引准备完毕之后，RAG应用需要在数据查询时检索数据并协助生成。

(1) **检索 (Retrieval)**：检索是借助数据索引从存储库中查找出相关的知识块，并按照相关性进行排序。经过排序的知识块将作为参考上下文用于协助后续的生成。

(2) **生成 (Generation)**：将检索阶段输出的相关知识块与用户原始的查询问题构造为 Prompt，输入给 LLM，LLM 根据 Prompt 生成内容。

随着 RAG 不断演进与优化，一些新的处理阶段被纳入流程，典型的两个阶段为检索前处理和检索后处理：

(1) **检索前处理 (Pre-Retrieval)**：检索前处理通常用于完成如查询转换、查询扩充、检索路由等处理工作，为后续检索和检索后处理做必要的准备，提高检索阶段召回知识的精确度和最终生成的质量。

(3) **检索后处理 (Post-Retrieval)**：在完成检索后对检索出的相关知识块做必要的补充处理，比如对检索结果使用更专业的排序模型与算法进行重排序，或过滤掉一些不符合条件的知识块，留下最需要最合规的知识块在上下文的最前端，协助 LLM 输出高质量内容。

1.3 RAG 与微调

在提升 LLM 效果的方法中，RAG 和微调是两种主流的方法。下面是这两种方法的比较：

比较内容	RAG	微调
动态知识更新	知识更新时只需动态更新知识库。	知识更新时需要重新准备训练数据对模型进行微调，迭代周期长。
训练成本	无需额外的训练成本。	需要额外的数据准备、标注、清洗成本，以及必要的算力与训练成本。
可解释性	可以追溯到具体的数据来源，有较好的可解释性和可追踪性。	可解释性较差。
幻觉	根据检索到的真实信息回答，降低了出现幻觉的概率。	使用特定领域的数据微调有助于减少幻觉的产生，但是无法完全杜绝幻觉。过度微调甚至可能导致模型产生新的幻觉，甚至严重的灾难性遗忘。
模型能力	模型自身不掌握特定知识，需要依赖外部知识库来支持检索和维护数据。	模型自身掌握特定知识，对下游应用更友好，在特定任务中使用更简单。
响应性能	有额外的检索步骤，耗时较长。	模型直接生成，耗时较短。
推理成本	输入带有上下文的、较长的 Prompt 会带来较高的推理成本。	无需向 Prompt 注入额外的知识，能够节约推理阶段使用的 token。

在实际场景中需要根据应用所需的侧重点选择使用何种技术，在条件允许的情况下，也可以将 RAG 和微调结合起来使用。

1.4 RAG 应用场景

- 企业知识库问答。
- 技术文档智能助手。
- 法律/医疗领域专业咨询。
- 个性化内容推荐。

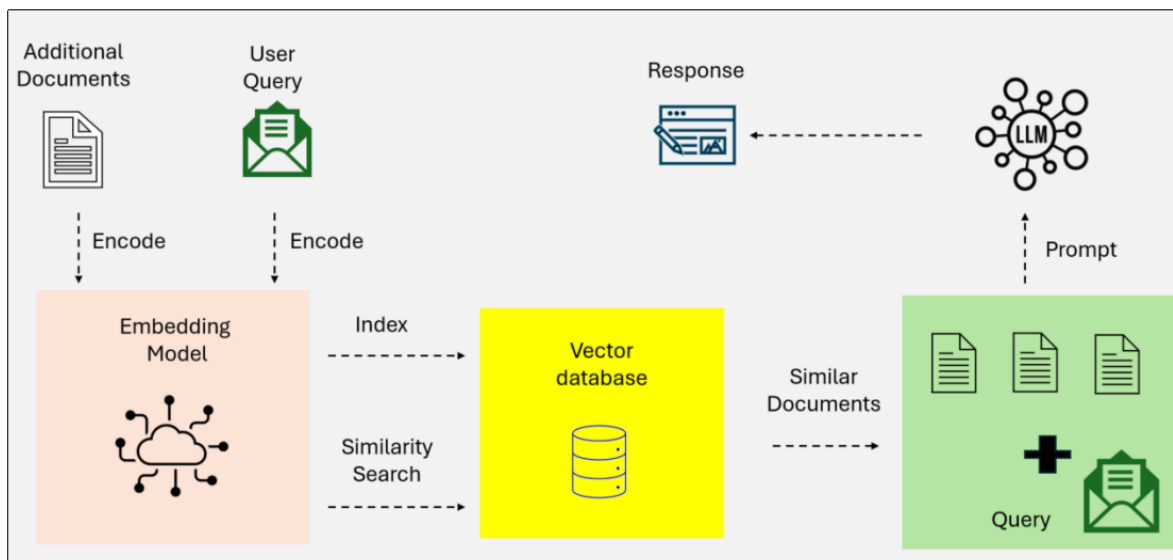
二、RAG 架构演进

RAG 最早在2020年由 Meta 的技术团队在 **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks** 中正式提出，但是在2022年 ChatGPT 出现之前，RAG 并没有获得过多关注。

ChatGPT 的出现使得 RAG 的研究迎来了拐点，大量研究开始关注如何使用 RAG 来提升 LLM 可控性并解决 LLM 在特定领域任务中的幻觉问题，优化 LLM 的推理和生成。

随着 GPT-4、Gemini Pro 等更先进的模型的出现，RAG 在更多任务场景中得以应用，随之而来的是传统 RAG 在实际应用中的问题不断凸显，因此更多的 RAG 范式和优化不断涌现。

2.1 Naive RAG (朴素RAG)



朴素 RAG 范式代表了最早的方法论，这一阶段的 RAG 遵循传统的基础流程，包括3个主要的模块与阶段：索引、检索和生成。

朴素RAG面临着诸多的问题：

1) 检索召回精度

检索阶段通常会面对精确度和召回率的挑战。如果精确度低，可能导致检索出来的内容带有无关甚至互相矛盾的数据；如果召回率低，可能导致检索出来的数据不够完整，缺失关键信息。

- 精确度 (Precision)：检索结果的“相关性”
$$\text{精确度} = (\text{检索到的相关文档数}) / (\text{检索到的总文档数})$$
- 召回率 (Recall)：检索结果的“完整性”
$$\text{召回率} = (\text{检索到的相关文档数}) / (\text{知识库中所有相关文档总数})$$

2) 大模型自身能力

对于检索出的信息，LLM 是否有能力推理、识别并区分，过滤掉无关或矛盾数据；能否使用检索内容准确生成问题的答案；能否避免在生成内容中包含检索出来的敏感或有害信息，也决定了最终生成结果的质量高低。

3) 数据增强的障碍

如何将检索到的信息整合起来生成答案也是一项颇具挑战的任务，有时可能出现生成的内容不连贯、脱节；或者存在多份相似检索内容时生成重复冗余内容的答案；也需要注意要对不同的检索内容的表述方式进行统一，生成风格一致的答案；也需要避免完全照搬检索到的内容，而没有进行信息的整合与处理。

4) 上下文大小限制

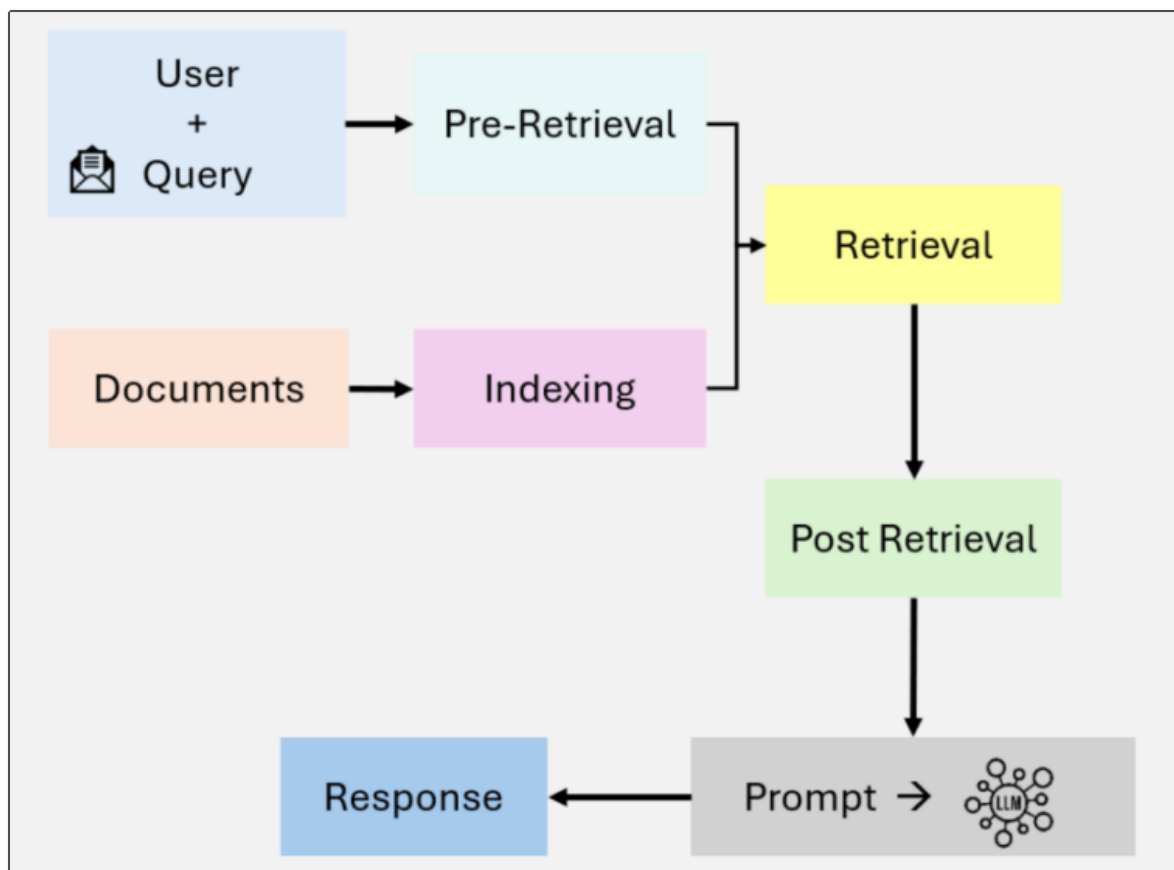
模型存在上下文大小限制，如果检索出的知识块过多，有可能导致留给生成的 token 过少无法完整生成内容；如果检索出的知识块内容过少，就有可能遗漏一些关键信息，无法让 LLM 完整了解所需知识，导致生成质量下降。因此，如何根据上下文大小确定知识块数量也是需要注意的问题之一。

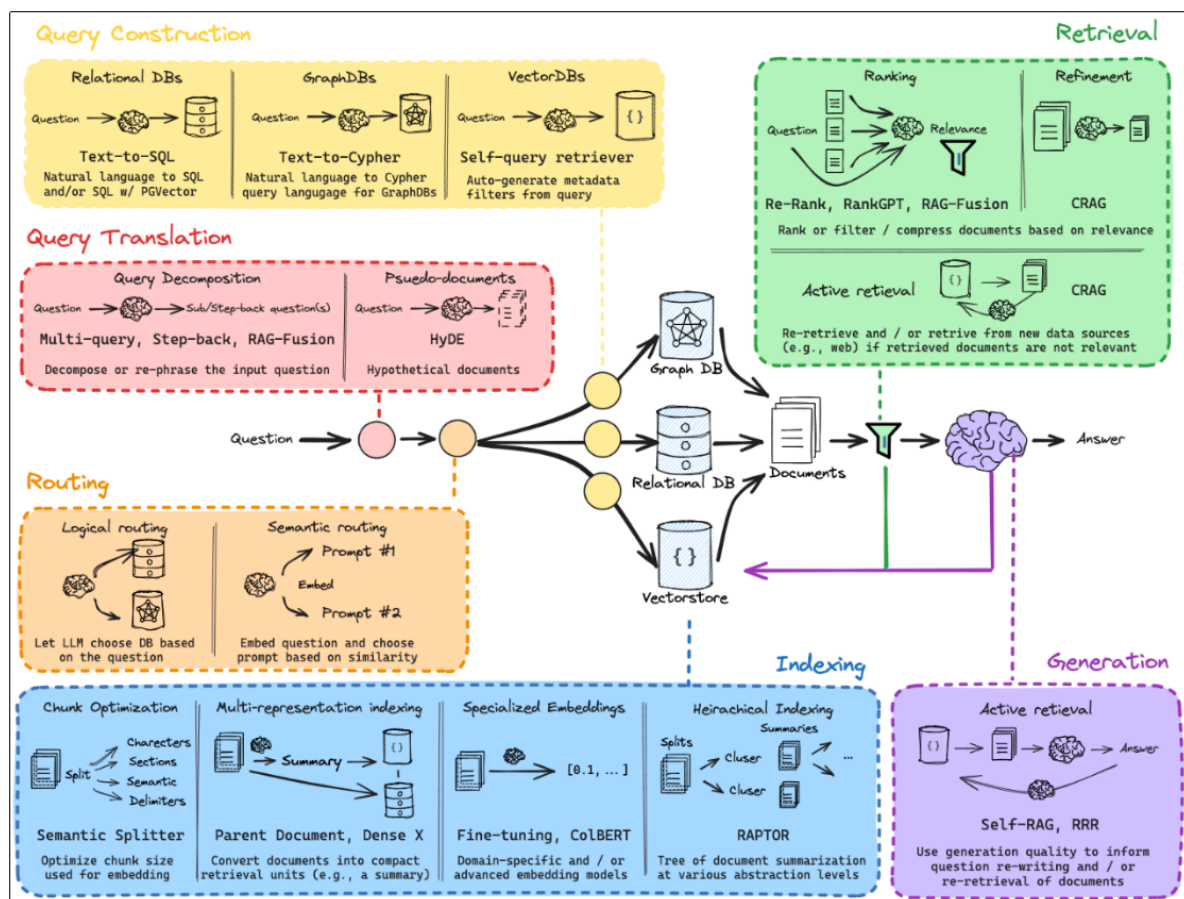
5) 响应性能

与 LLM 直接输出相比，RAG 应用增加了更多处理步骤，并且随着 RAG 的改进优化，未来可能存在更多的步骤，这也意味着端到端的响应性能会下降。如何权衡最终输出质量和响应性能也是一个问题。

2.2 Advanced RAG（高级RAG）

高级 RAG 引入了许多改进来提高检索的质量，主要包含检索前处理和检索后处理两大部分。





2.2.1 检索前处理

(1) 优化分块策略

常见的分块策略是将文档按固定数量的 token 拆分成块。较大的块能够捕获更多的上下文，但是会产生更多的噪声，并且需要更长的处理时间；较小的块可能无法完整包含所需上下文，导致关键信息被截断。

这促使人们优化递归拆分和滑动窗口方法，通过合并多个检索过程中的全局相关信息来实现分层检索。然而，这些方法仍然无法在语义完整性和上下文长度之间取得平衡。

因此之后又提出了类似 Small2Big 的方法，其中句子（小）用作检索单位，并将前后句子作为（大）上下文提供给 LLM。

(2) 元数据索引

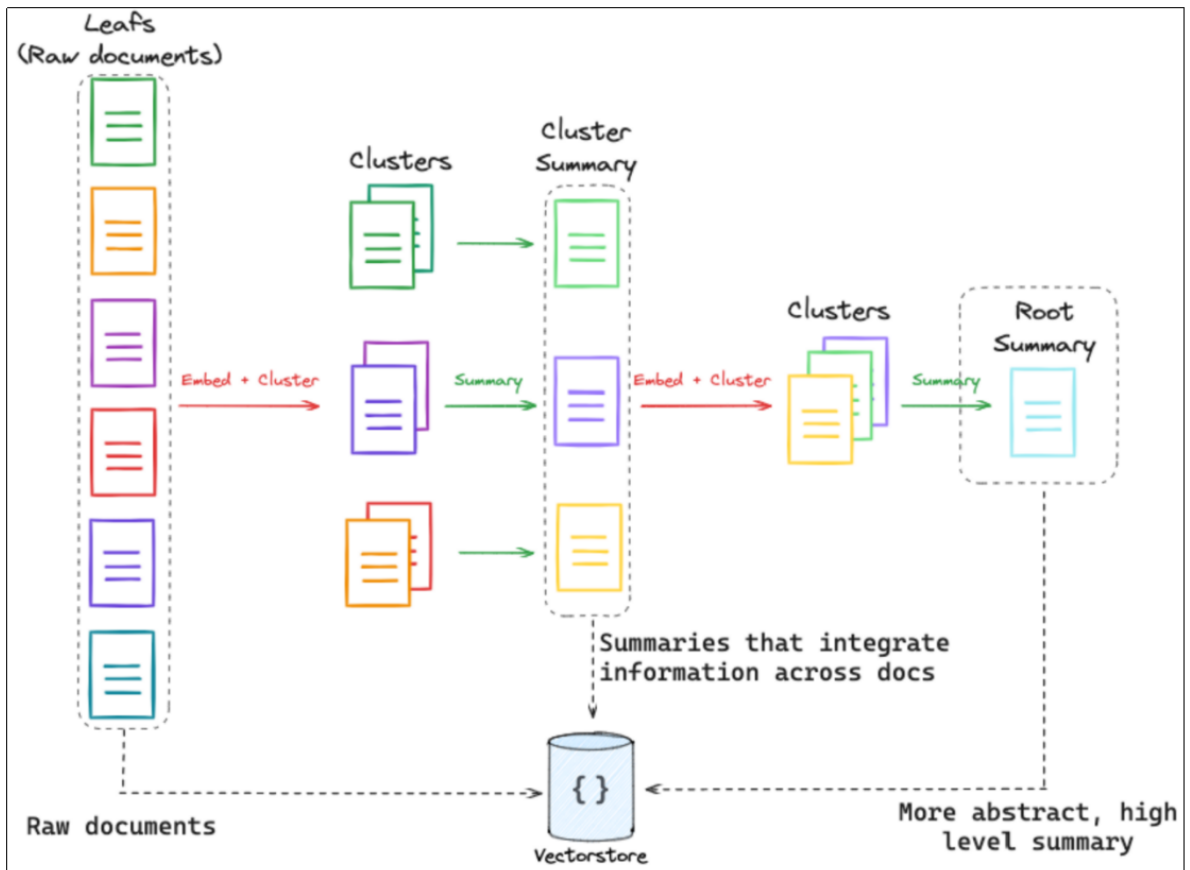
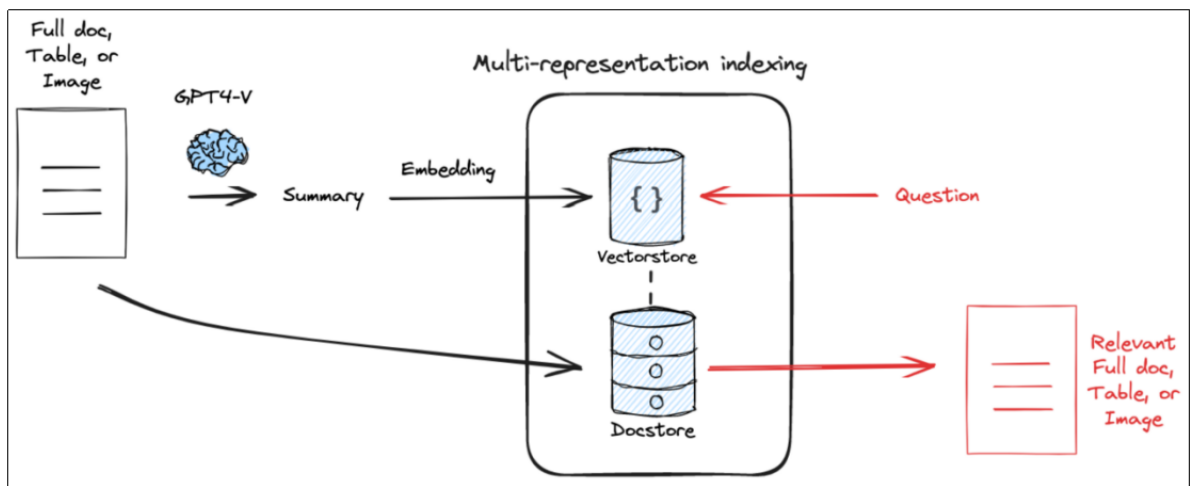
可以使用元数据信息（例如页码、文件名、作者、类别时间戳）来丰富数据块。并根据这些元数据过滤检索，从而限制检索范围。

在检索过程中为文档时间戳分配不同的权重可以实现时间感知的检索关联分析，从而确保知识的新鲜度并避免信息过时。

除了从原始文档中提取元数据外，还可以人为构建元数据。例如，添加段落摘要，以及引入假设性问题。这种方法也称为“逆向 HyDE”（Reverse HyDE）：具体来说，利用 LLM 生成文档可以回答的问题，然后在检索过程中计算原始问题与假设问题的相似度，以缩小问题与答案之间的语义差距。

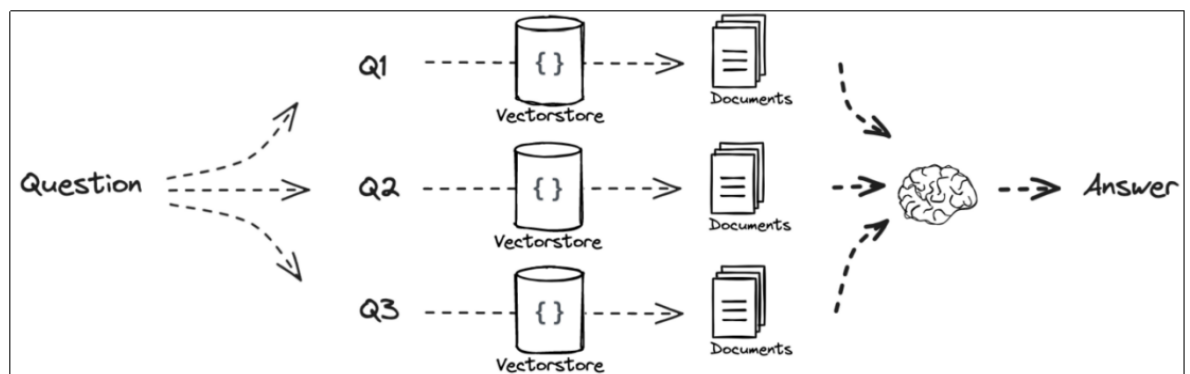
(3) 结构化索引

建立文档的层次结构，来加快 RAG 系统检索相关数据的速度。



(4) 查询扩展

将单个长度较短，语义不完整的查询扩展为多个查询来丰富查询的内容，提供更完整的上下文来弥补原始问题中缺乏的细节，从而确保生成的答案具有最佳相关性。

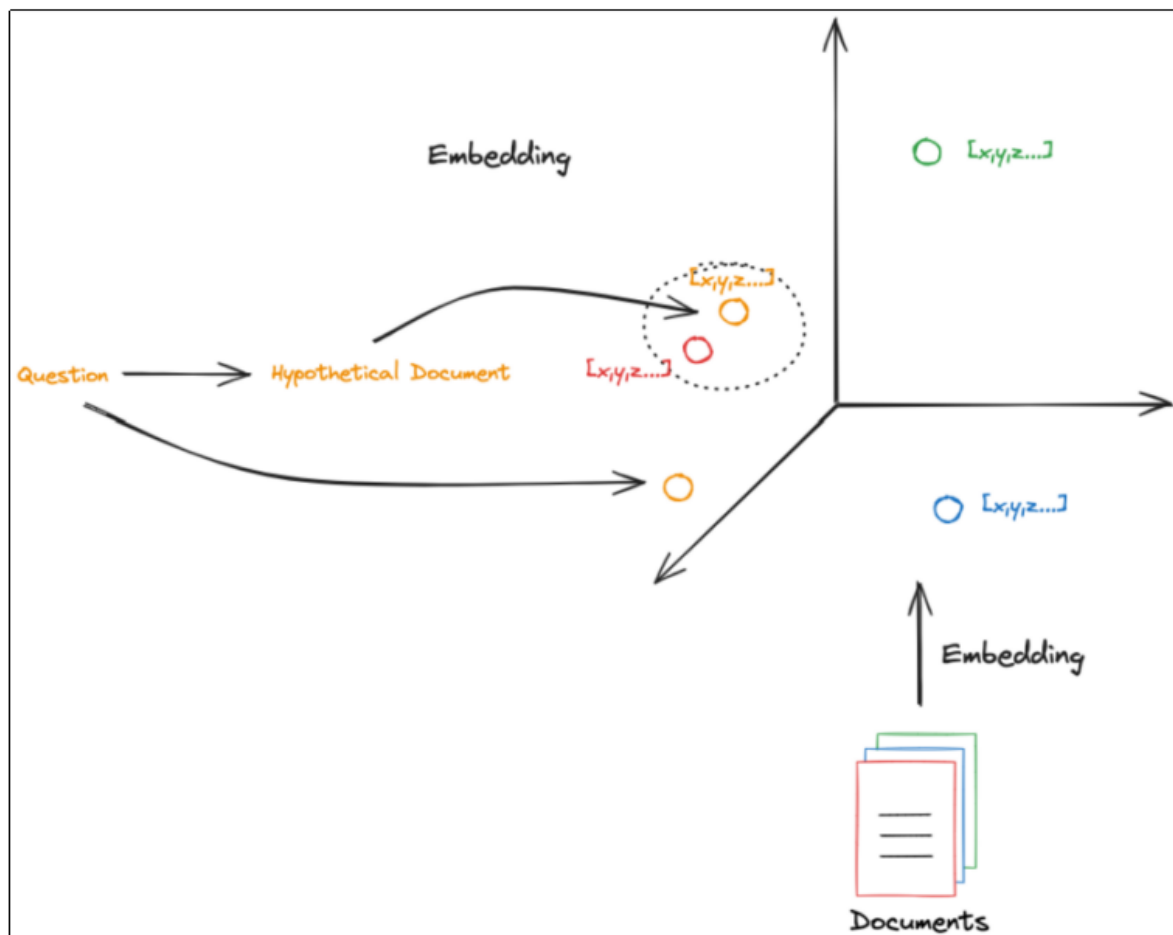


(5) 查询重写

原始的查询未必最适合检索，可以使用 LLM 对查询内容进行重写之后在进行检索，也可以使用专门训练的小语言模型来重写查询。

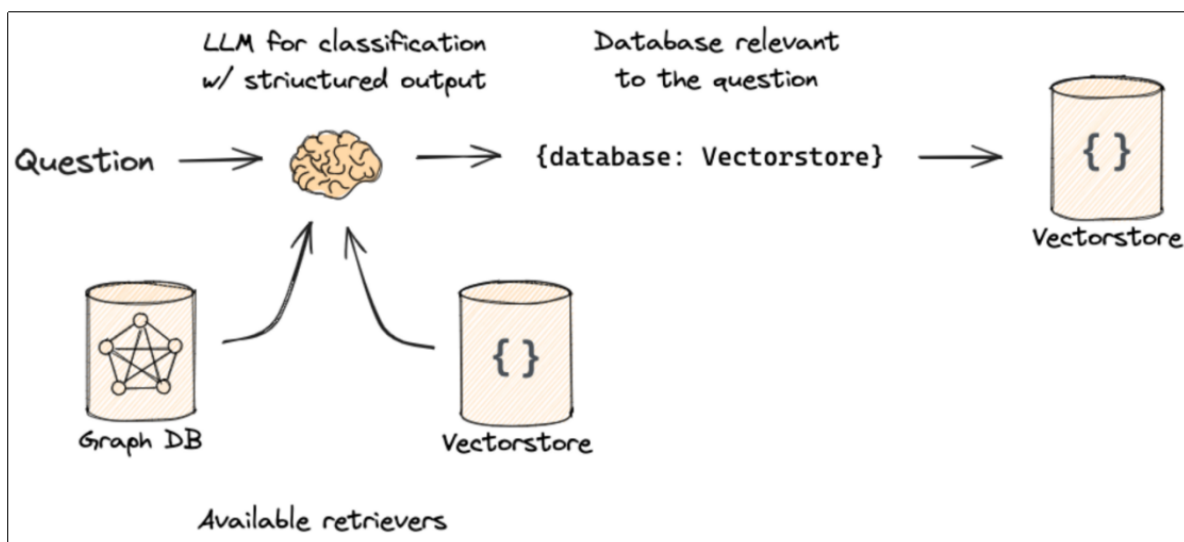
(6) HyDE

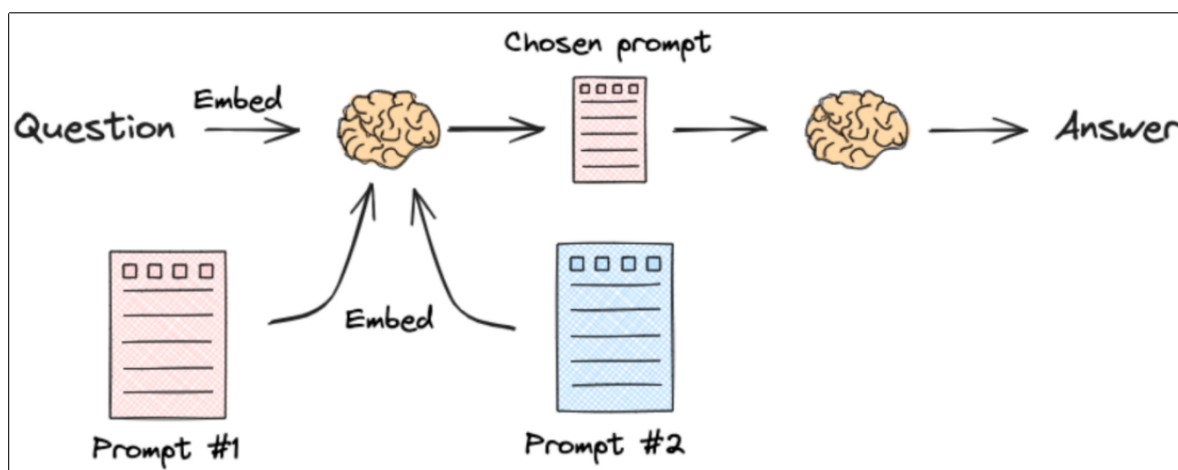
Hypothetical Document Embeddings 是一种基于假设文档的检索方法，会先生成一个假设的答案文档，使用这个假设文档进行向量化检索，计算答案与答案之间的相似度，而非查询与答案之间的相似度。



(7) 查询路由

根据用户查询的特征，将其路由到不同的知识源、检索策略或子模型。提升 RAG 系统的灵活性、效率和回答质量。





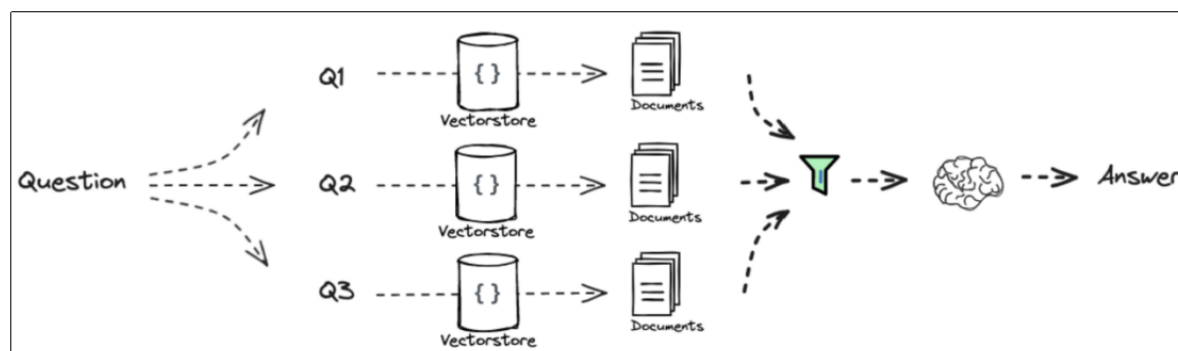
(8) 混合检索

结合稀疏检索（关键词匹配）和稠密检索（向量相似度）来捕捉不同的相关特征，利用互补的相关性信息来增益检索。

2.2.2 检索后处理

(1) 重排序

将初步检索得到的结果进行重新排序来突出显示最相关的信息，为模型提供更精炼更准确的信息。重排序可以使用基于规则的方法，根据预定义的指标进行重排序；也可以使用基于模型的方法进行重排序。



(2) 上下文压缩

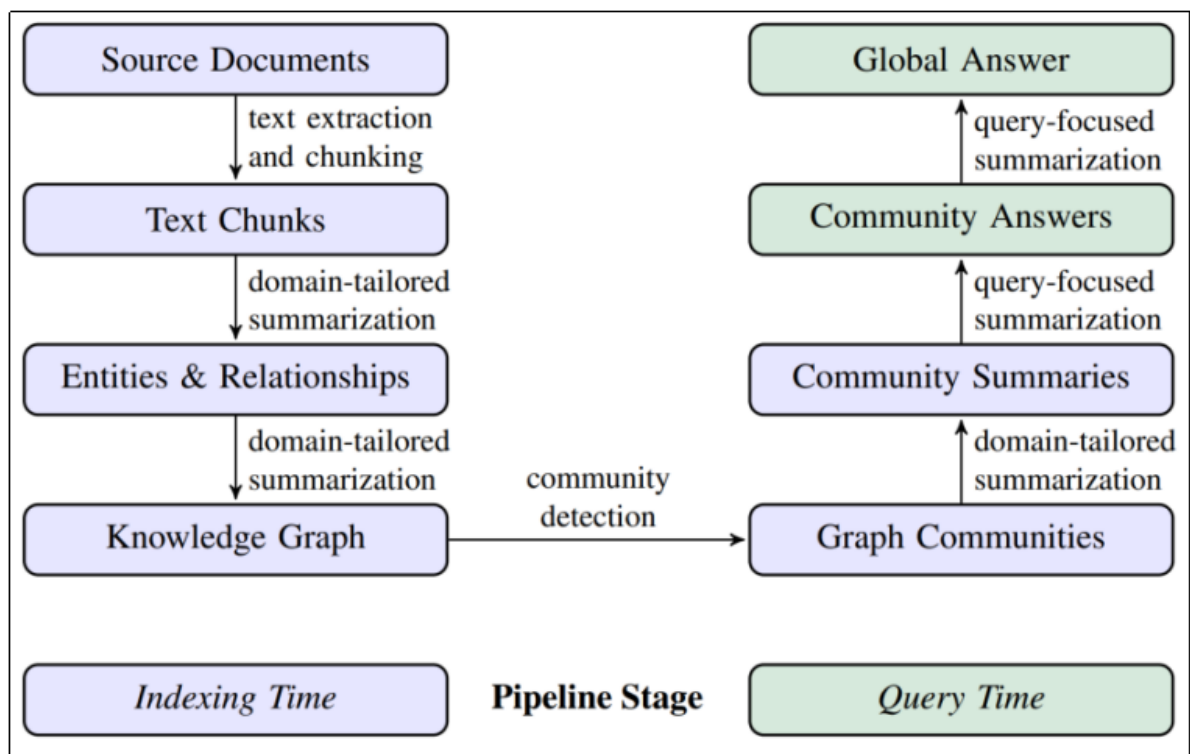
过多的上下文可能包含大量噪声，削弱 LLM 对关键信息的感知。上下文压缩会对检索到的多个文档或片段进行处理，在不丢失重要信息的前提下压缩其长度和数量，来更好地适应模型上下文大小并提高信息质量。压缩方式包括剔除不重要 token、提取关键句、生成段落摘要等方式。

2.3 Graph RAG

Graph RAG 是一种结构化、分层的 RAG 方法，与使用纯文本片段的朴素语义搜索方法不同，Graph RAG 的过程包括从原始文本中提取知识图谱，构建社区层次结构，为这些社区生成摘要，然后利用这些结构执行 RAG 任务。

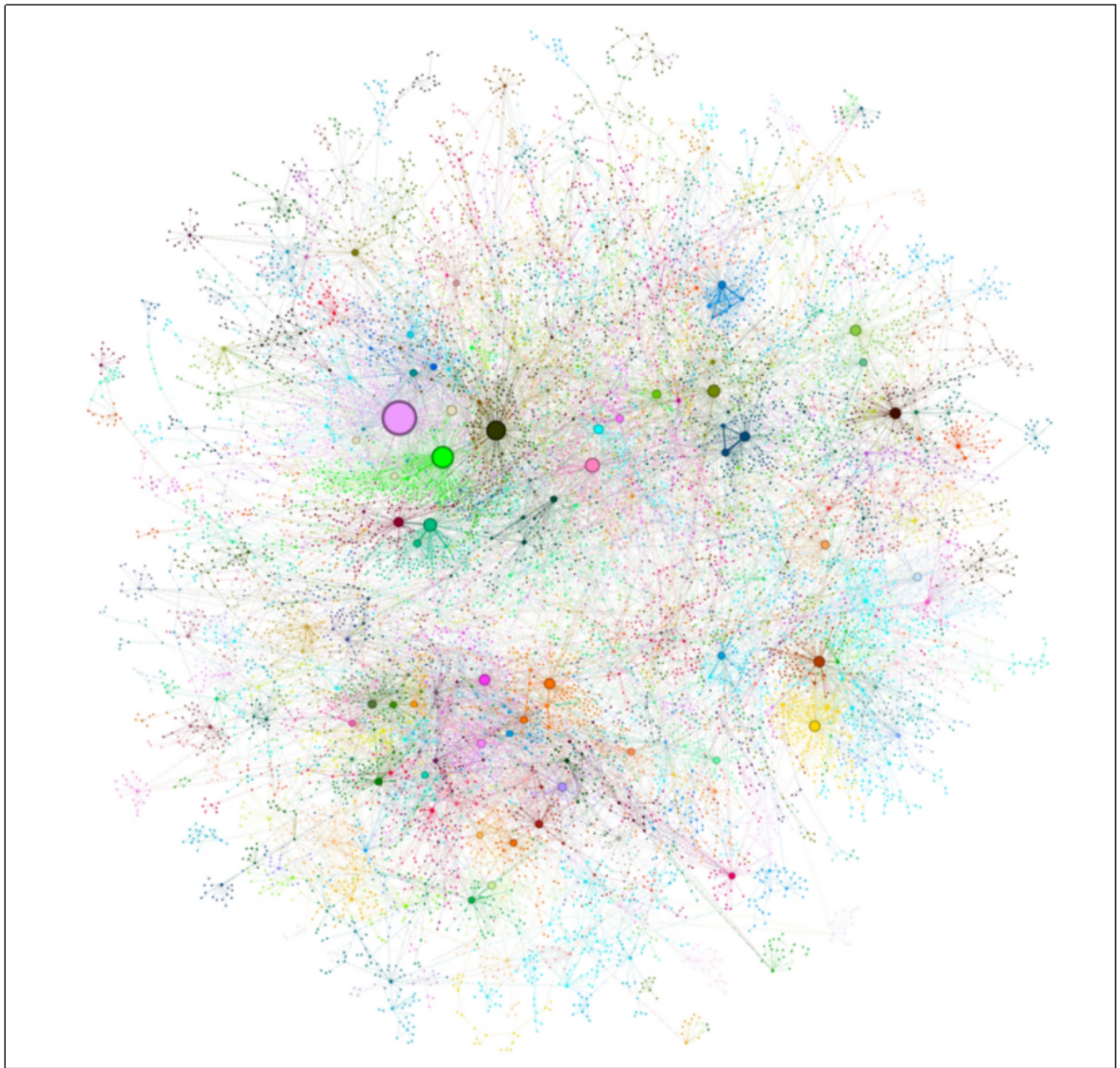
基线 RAG 在需要通过遍历不同的信息片段来提供新的综合性见解的时候，难以将信息联系起来。在被要求全面理解大型数据集或单个大型文档中的总结语义概念时，基线 RAG 的表现也不尽如人意。因此，新的 Graph RAG 方法使用 LLM 为数据集创建知识图谱，将该图谱和图机器学习一起使用，在进行复杂信息文档分析时显著提升了问答性能。

Graph RAG 工作的基本流程如下：



(1) LLM 处理数据集，创建对源数据中所有实体和关系的引用，并根据这些实体和关系创建一个知识图谱。

(2) 对图谱自底向上进行聚类，将数据分层组织成语义簇（社区），并根据划分预先总结语义概念和主体，这有助于对数据集进行整体理解。下图是一个图结构的可视化，每个圆圈代表一个实体，实体的大小代表该实体关系数量的多少，颜色代表实体属于哪一个社区。



(3) 在查询时，实体和社区都会用于为 LLM 提供上下文材料，使之能够在不同的抽象层次上回答问题。

2.4 Agentic RAG

1) Agentic RAG 的特点

传统的 RAG 系统，因其静态的工作流程和有限的适应性，往往难以处理动态的多步骤推理和复杂的现实世界任务。这些局限性促使了 Agentic RAG 的出现。Agentic RAG 通过引入能够进行动态决策和工作流优化的智能体，来处理复杂、实时和多领域的查询，实现了从静态系统到动态系统的范式转变。

Agentic RAG 的主要特点包括：

- 自主决策：智能体根据查询的复杂性独立评估和管理检索策略。
- 迭代优化：引入反馈循环以提高检索准确性和响应相关性。
- 工作流优化：动态协调任务，实现实时应用。

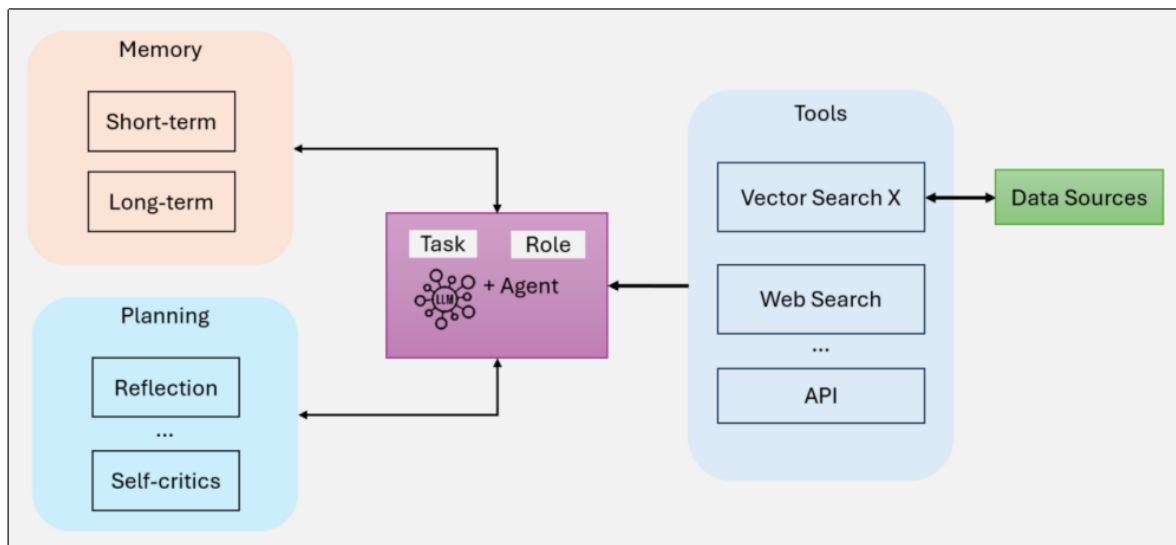
Agentic RAG 目前仍然面临一些问题：

- 协调复杂性：管理智能体之间的交互需要复杂的协调机制。
- 计算开销：使用多个智能体增加了复杂工作流程的资源需求。
- 可扩展性限制：虽然系统具有可扩展性，但其动态特性在高查询量时可能对计算资源造成压力。

2) Agent 组成

一个智能体包括4个组成部分，分别是：

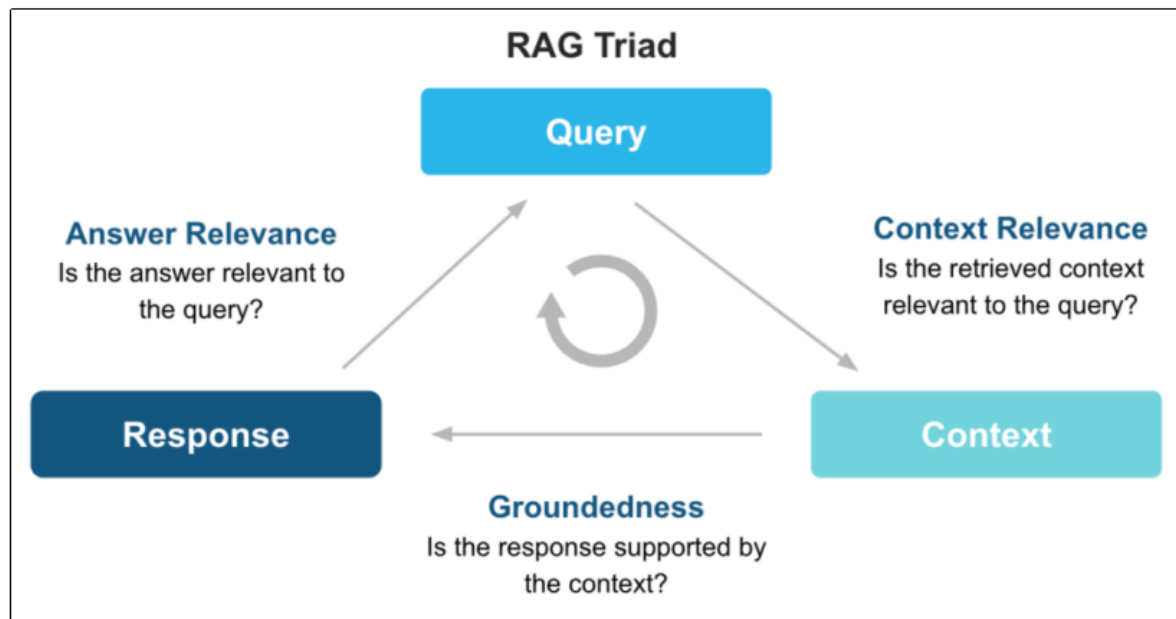
- LLM：作为智能体的主要推理引擎，负责解释查询，生成响应，并保持对话的连贯性。
- 记忆：捕捉跨交互的上下文和相关信息，分为短期记忆和长期记忆。短期记忆跟踪即时对话状态，长期记忆存储积累的知识和经验。
- 规划：通过反思、查询路由或自我批评，指导智能体的迭代推理过程，确保复杂任务被有效分解和执行。
- 工具：扩展智能体文本生成之外的能力，使其能够访问外部资源、实时数据或进行专业计算。



三、RAG 评估

RAG 的评估主要包含三个主要质量分数和若干个基本能力，这些评估指标共同构成了对 RAG 两个主要目标的评估：检索和生成。

3.1 质量分数评估



质量分数包括：

1) 上下文相关性 (Context Relevance)

评估检索到的上下文与查询之间的相关性。

2) 答案真实性 (Groundedness)

评估生成的答案是否能被检索内容所支撑。

3) 答案相关性 (Answer Relevance)

评估生成的答案与查询之间的相关性。

3.2 基本能力评估

RAG 评估还包括许多表明其适应性和效率指标：

1) 抗噪声能力 (Noise Robustness)

评估模型处理与问题相关但缺乏实质性信息的噪声文档的能力。

2) 负向拒绝 (Negative Rejection)

评估模型在检索到的文档不包含回答问题所需知识时拒绝回答的能力。

3) 信息整合 (Information Integration)

评估模型从多个文档中综合信息以回答复杂问题的能力。

4) 反事实鲁棒性 (Counterfactual Robustness)

评估模型识别并忽略文档中已知错误的能力。

3.3 评估工具 Ragas

Ragas 是一个开源的、用于评估 RAG 系统质量的端到端评估框架，使用基于 LLM 和传统指标精确评估 RAG 系统中的检索质量、生成质量以及二者之间的耦合关系。

Ragas 包含多种评估指标，包括无参考指标和有参考指标：

1) 检索阶段

上下文精确度 (Context Precision)：检索内容中相关信息的比例。

上下文召回率 (Context Recall)：检索内容中是否遗漏相关信息。

上下文实体召回率 (Context Entities Recall)：检索内容中相关实体是否被遗漏。

上下文相关性 (Context Relevancy)：上下文与问题的相关性。

2) 生成阶段

响应真实性 (Response Groundedness)：生成内容是否有上下文支撑，是否存在臆造。

忠诚度 (Faithfulness)：生成内容与上下文的一致性，是否存在曲解。

响应相关性 (Response Relevancy)：生成内容与问题的相关性。

噪声敏感度 (Noise Sensitivity)：生成内容时抵抗噪声的能力。

答案准确度 (Answer Accuracy)：生成内容与标准答案的一致性。

事实正确性 (Factual Correctness)：生成内容与标准答案的事实重叠。

语义相似度 (Semantic Similarity)：生成内容与标准答案的语义相似程度。